CSCI 611 Final Project Report

Credit Card Fraud Detection Using Machine Learning and Neural Networks

Team Members: Jayana Sarma, Nayana Nagarajappa

1. Introduction

Credit card fraud detection is an important real-world machine learning problem. Financial institutions process large numbers of transactions every day, and fraudulent transactions can lead to major financial losses. The goal of this project was to build a system that can classify credit card transactions as either legitimate or fraudulent.

This problem is challenging because fraudulent transactions are very rare compared to legitimate transactions. In the original dataset, only 492 out of 284,807 transactions were fraudulent. This means fraud cases make up less than 0.2% of the full dataset. Because of this severe class imbalance, accuracy alone is not a reliable evaluation metric. A model can achieve very high accuracy by predicting almost every transaction as legitimate while still missing most fraud cases.

The purpose of this project was to compare several machine learning and neural network models for fraud detection, evaluate them using appropriate metrics, tune decision thresholds, and build a simple Streamlit app for demonstration.

The models compared in this project were:

- Logistic Regression
- Balanced Logistic Regression
- Random Forest
- Random Forest with threshold tuning
- XGBoost
- Feedforward Neural Network
- Deep Neural Network
- Deep Neural Network with Dropout

The final selected model was Random Forest with threshold 0.3, because it achieved the best F1-score and provided the best balance between catching fraud cases and avoiding too many false alarms.

2. Overall Context and Relevant Work

Fraud detection is a common application of supervised machine learning. Traditional fraud detection systems often use manually written rules, but fraud patterns can change over time. Machine learning models can learn patterns from historical transaction data and use those patterns to detect suspicious transactions.

However, fraud detection has several challenges:

- Fraud cases are rare.
- False negatives can be costly because they represent missed fraud.
- False positives can create unnecessary alerts for legitimate customers.
- The best threshold for fraud detection may not be the default threshold of 0.5.
- Evaluation requires metrics beyond accuracy.

Previous machine learning approaches for structured tabular data often use models such as Logistic Regression, Random Forest, gradient boosting methods, and neural networks. Tree-based ensemble models such as Random Forest and XGBoost are commonly effective on tabular datasets because they can capture nonlinear relationships and feature interactions.

In this project, we compared both traditional machine learning models and neural network models. We also focused on threshold tuning because fraud detection depends heavily on the balance between precision and recall.

3. High-Level Framework of the Solution

The overall solution followed an end-to-end machine learning workflow.

Step 1: Data Collection

The Kaggle Credit Card Fraud Detection dataset was loaded and analyzed.

https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

This is the link for the dataset used

Step 2: Exploratory Data Analysis

We checked:

- dataset shape,
- column names,
- missing values,
- class distribution,
- amount distribution,

- and fraud imbalance.

Step 3: Data Preprocessing

The dataset was divided into training, validation, and testing sets using stratified sampling. Stratification was important because fraud cases are rare, and each split needed to preserve the same fraud-to-legitimate ratio.

Numerical features were scaled using StandardScaler for models that are sensitive to feature magnitude, such as Logistic Regression and Neural Networks.

Step 4: Baseline Modeling

Initial baseline models were trained:

- Logistic Regression
- Balanced Logistic Regression

These models provided reference performance.

Step 5: Advanced Modeling

More advanced models were trained:

- Random Forest
- XGBoost
- Feedforward Neural Network
- Deep Neural Network
- Deep Neural Network with Dropout

Step 6: Threshold Tuning

Decision thresholds were adjusted to improve the balance between precision and recall. Instead of relying only on the default threshold of 0.5, we evaluated threshold-based predictions such as 0.3 for Random Forest.

Step 7: Model Evaluation

Models were compared using:

- Precision
- Recall
- F1-score
- PR-AUC
- ROC-AUC
- Confusion matrices

Step 8: Streamlit App Demonstration

The final model was saved and integrated into a Streamlit app that allows users to upload transaction data and receive fraud predictions.

4.Unique Aspects of the Solution

The uniqueness of this project is not only that multiple models were trained, but that the final model was selected using threshold-aware evaluation. Since the dataset is highly imbalanced, accuracy was not enough to judge model quality.

The project focused on:

- comparing multiple model families,
- analyzing neural network architectures,
- tuning thresholds,
- comparing F1-score and PR-AUC,
- saving the model and preprocessing files,
- and building a working Streamlit demo app.

Another important part of the solution was preventing training-serving mismatch. The app loads the saved feature names and reorders uploaded CSV columns before prediction. This ensures that the input data used in the app matches the format used during model training.

5. Dataset Description

The dataset used in this project was the Kaggle Credit Card Fraud Detection dataset.

The dataset contains:

- 284,807 total transactions
- 492 fraudulent transactions
- 284,315 legitimate transactions

The target column is:

- `Class = 0`: legitimate transaction
- `Class = 1`: fraudulent transaction

The input features are:

- `Time`
- `Amount`
- `V1` through `V28`

The `V1` through `V28` features are anonymized PCA-transformed variables. Because of this anonymization, the original meaning of each feature is not available. However, these features still contain useful patterns for fraud classification.

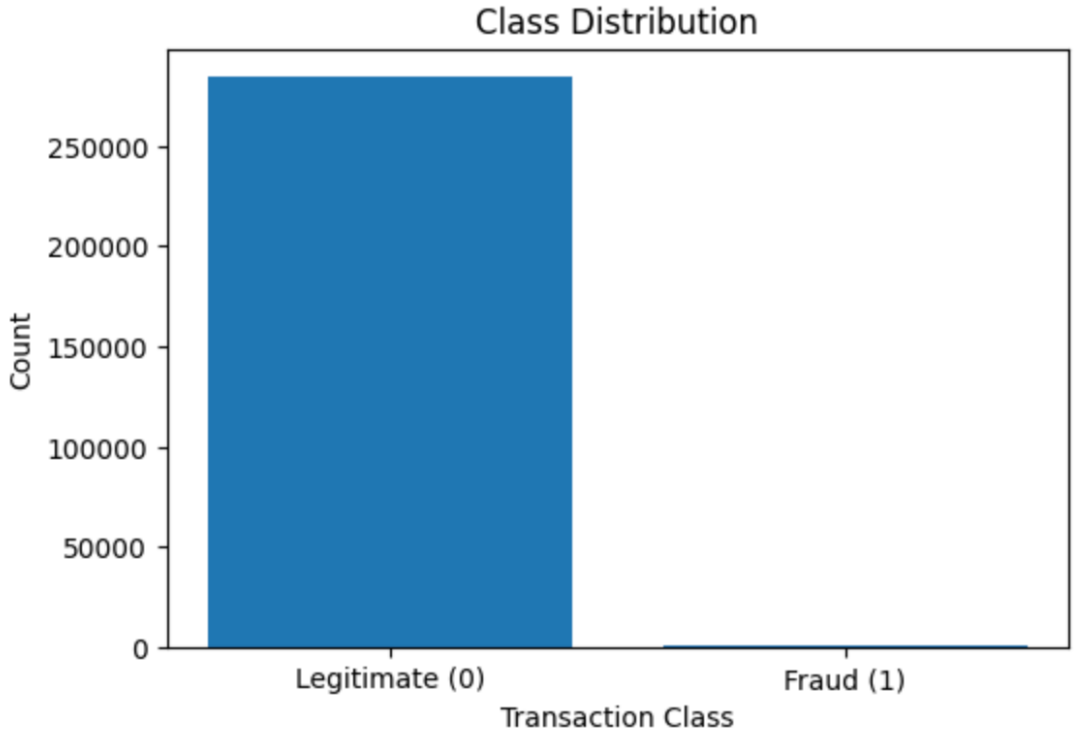


Figure 1. Class distribution showing the severe imbalance between legitimate and fraudulent transactions.

6. Data Preprocessing

6.1 Train, Validation, and Test Split

The dataset was split into training, validation, and testing sets. A stratified split was used to preserve the class distribution across all three sets.

```
X = df.drop('Class', axis=1)
y = df['Class']

# Step 1: train+val and test
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Step 2: train and validation
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp
)
```

The resulting split sizes were:

- Training set: 170,883 rows
- Validation set: 56,962 rows
- Test set: 56,962 rows

The class distribution was preserved across the splits. The training set had approximately 99.83% legitimate transactions and 0.17% fraud transactions. The test set had a very similar distribution.

6.2 Feature Scaling

Feature scaling was applied using StandardScaler.

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

Scaling was important for Logistic Regression and Neural Networks because these models are sensitive to feature magnitude. Random Forest does not strictly require scaling, but the scaler was still saved for the app pipeline because other models and app preprocessing depended on consistent input handling.

6.3 Class Imbalance Handling

The dataset was highly imbalanced, so several strategies were used:

- Balanced Logistic Regression
- threshold tuning

- PR-AUC evaluation
- F1-score-based final model selection
- class weights in neural network experiments

Balanced Logistic Regression increased recall, but it also produced many false positives. Threshold tuning helped improve the balance between recall and precision.

7. Machine Learning Models and Architectures

A major goal of the project was to compare different model families and understand how model construction affected fraud detection performance.

7.1 Logistic Regression

Logistic Regression was used as the first baseline model. It is simple, interpretable, and commonly used for binary classification.

The model outputs a probability using the sigmoid function. A threshold is then applied to convert the probability into a class prediction.

Configuration:

Parameter	Value
Model	Logistic Regression
Max iterations	1000
Input data	Scaled features
Threshold evaluated	0.3

The Logistic Regression model gave reasonable precision but lower recall compared with stronger models.

7.2 Balanced Logistic Regression

Balanced Logistic Regression was used to handle class imbalance by giving more weight to the minority fraud class.

```
model_balanced = LogisticRegression(class_weight='balanced', max_iter=1000)
model_balanced.fit(X_train_scaled, y_train)
```

This model achieved high recall, meaning it caught many fraud cases. However, it also produced too many false positives, which caused precision to drop significantly. This showed that class weighting alone was not enough to create the best fraud detection model.

7.3 Random Forest

Random Forest was the strongest model in this project. It is an ensemble model that combines many decision trees and averages their predictions.

The final Random Forest model used in the notebook was:

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)

rf_model.fit(X_train, y_train)
```

Random Forest worked well because:

- it handles nonlinear relationships,
- it performs strongly on tabular datasets,
- it is robust to noisy features,
- and it gave a strong precision-recall balance after threshold tuning.

The default Random Forest model had high precision but slightly lower recall. After applying a threshold of 0.3, recall improved and the model achieved the best F1-score.

7.4 XGBoost

XGBoost was added as an advanced boosting model. XGBoost builds trees sequentially, where each new tree attempts to correct errors made by previous trees.

The model used class imbalance information through `scale_pos_weight`.

```
num_legit = (y_train == 0).sum()
num_fraud = (y_train == 1).sum()

scale_pos_weight = num_legit / num_fraud

print("Number of legitimate transactions:", num_legit)
print("Number of fraudulent transactions:", num_fraud)
print("scale_pos_weight:", scale_pos_weight)
```

```
Number of legitimate transactions: 170588
Number of fraudulent transactions: 295
scale_pos_weight: 578.264406779661
```

```
xgb_model = XGBClassifier(
    n_estimators=300,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=scale_pos_weight,
    eval_metric="logloss",
    random_state=42,
    n_jobs=-1
)

xgb_model.fit(X_train_scaled, y_train)
```

XGBoost achieved strong PR-AUC and performed well overall. However, Random Forest with threshold 0.3 achieved a slightly better F1-score.

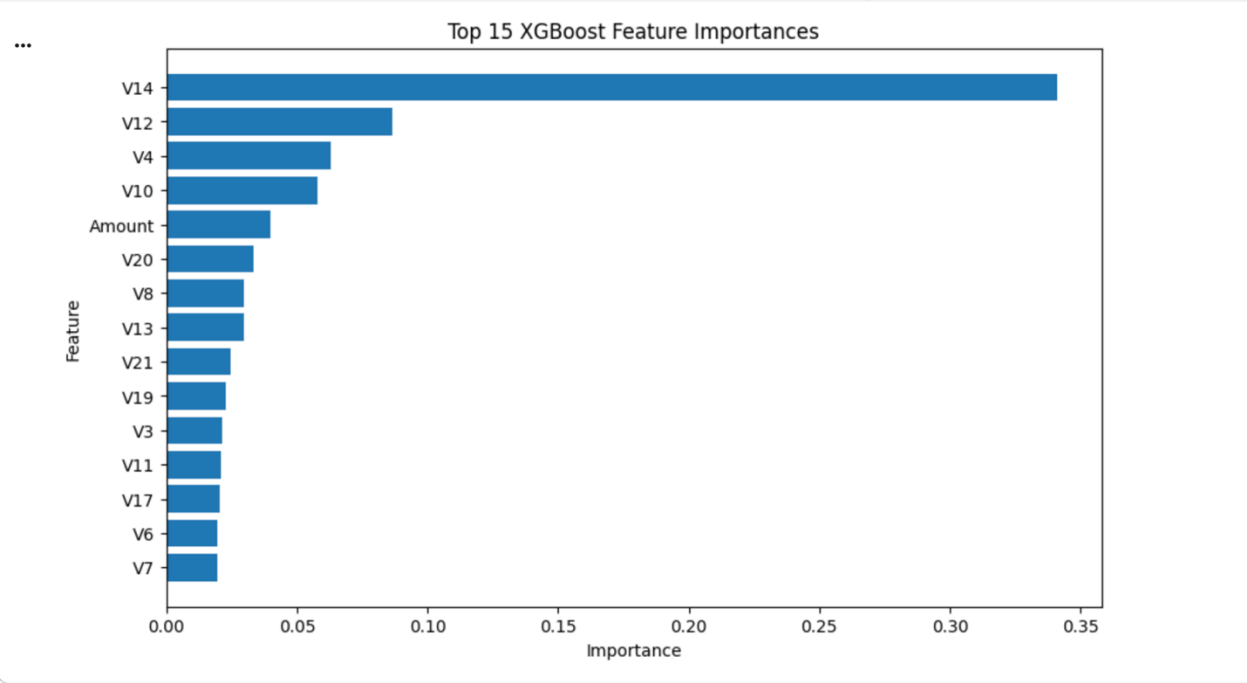


Figure: XGBOOST Feature Importance

7.5 Neural Network Models

Neural networks were included because the project focused on both machine learning and neural network-based classification.

The neural network models used feedforward multilayer perceptron architectures with ReLU activation in hidden layers and sigmoid activation in the output layer.

The sigmoid output was used because this is a binary classification problem.

7.5.1 Feedforward Neural Network

The first neural network used a simple feedforward architecture.

Architecture:

Layer	Neurons	Activation
Input	30	-
Dense 1	32	ReLU
Dense 2	16	ReLU
Output	1	Sigmoid

This model was useful as a neural network baseline. It achieved competitive recall but did not outperform Random Forest.

7.5.2 Deep Neural Network and Dropout Mode

To improve neural network performance, multiple architectures were compared:

- Shallow Neural Network
- Deep Neural Network
- Deep Neural Network with Dropout

```
def build_nn_model(model_type="shallow"):
    model = keras.Sequential()
    model.add(keras.Input(shape=(X_train_scaled.shape[1],)))

    if model_type == "shallow":
        model.add(layers.Dense(32, activation="relu"))

    elif model_type == "deep":
        model.add(layers.Dense(64, activation="relu"))
        model.add(layers.Dense(32, activation="relu"))
        model.add(layers.Dense(16, activation="relu"))

    elif model_type == "dropout":
        model.add(layers.Dense(64, activation="relu"))
        model.add(layers.Dropout(0.3))
        model.add(layers.Dense(32, activation="relu"))
        model.add(layers.Dropout(0.3))
        model.add(layers.Dense(16, activation="relu"))

    model.add(layers.Dense(1, activation="sigmoid"))

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.001),
        loss="binary_crossentropy",
        metrics=[
            "accuracy",
            keras.metrics.AUC(name="roc_auc"),
            keras.metrics.AUC(name="pr_auc", curve="PR")
        ]
    )

    return model
```

Dropout was used to reduce overfitting by randomly disabling some neurons during training. The deep neural network with dropout performed best among the neural network architectures, but it still did not outperform Random Forest with threshold tuning.

	Model	Threshold	Precision	Recall	F1-score	ROC-AUC	PR-AUC
2	Deep NN + Dropout	0.94	0.778846	0.826531	0.801980	0.959434	0.719262
1	Deep NN	0.95	0.711538	0.755102	0.732673	0.972468	0.695290
0	Shallow NN	0.95	0.537415	0.806122	0.644898	0.962181	0.631120

Neural network architecture comparison showing shallow, deep, and dropout-based models.

8. Threshold Selection and Decision Boundary Analysis

One of the most important parts of this project was threshold tuning.

Most classification models output probabilities. A threshold converts those probabilities into class predictions.

The default threshold is usually 0.5:

- probability >= 0.5 means fraud
- probability < 0.5 means legitimate

However, the default threshold was not always best for fraud detection. Since fraud is rare, adjusting the threshold can improve recall or precision depending on the goal.

For the final Random Forest model, threshold 0.3 gave the best balance between precision and recall.

```
y_test_prob_rf = rf_model.predict_proba(X_test)[:, 1]
y_test_pred_rf_03 = (y_test_prob_rf >= 0.3).astype(int)
```

At threshold 0.3, the Random Forest model achieved:

Metric	Value
Precision	0.8660
Recall	0.8571
F1-score	0.8615

Observed tradeoff:

- Lower thresholds increased recall but created more false positives.
- Higher thresholds increased precision but missed more fraud cases.

- Threshold 0.3 provided the best practical balance in the final model comparison.

9. Evaluation Metrics

Because the dataset was highly imbalanced, accuracy was not enough to evaluate performance.

The main evaluation metrics were:

Precision

Precision measures how many predicted fraud cases were truly fraud.

High precision means fewer false alarms.

Recall

Recall measures how many actual fraud cases the model detected.

High recall means fewer missed fraud cases.

F1-score

F1-score balances precision and recall.

This was used for final model selection because fraud detection requires a balance between catching fraud and avoiding too many false alarms.

PR-AUC

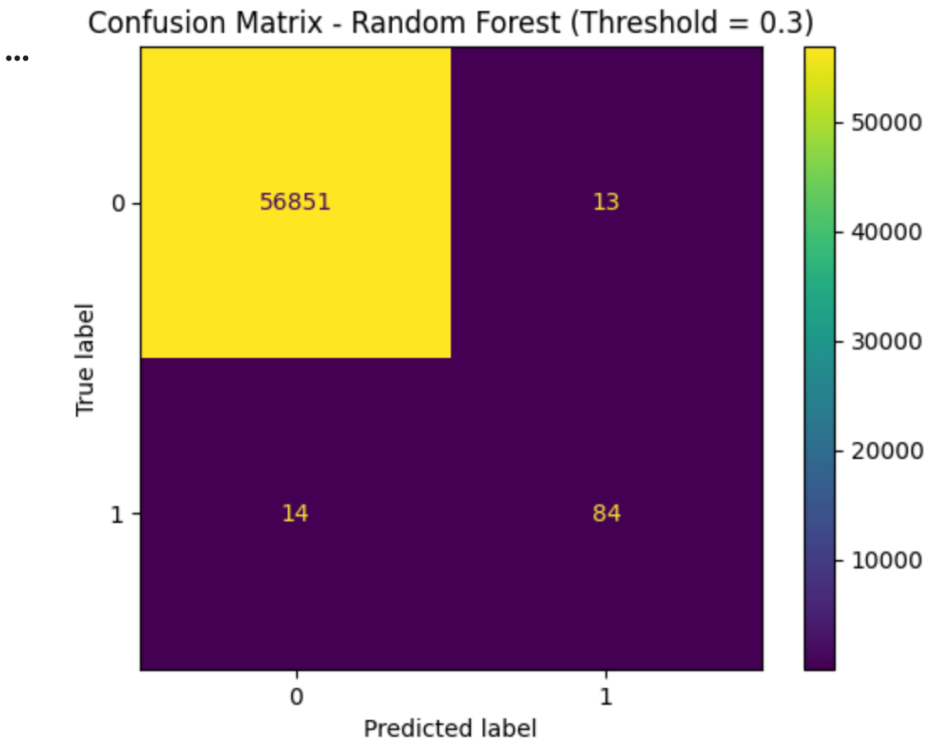
PR-AUC evaluates the precision-recall tradeoff across thresholds.

This is especially useful for imbalanced datasets where the positive class is rare.

Confusion Matrix

Confusion matrices were used to examine:

- true negatives,
- false positives,
- false negatives,
- and true positives.



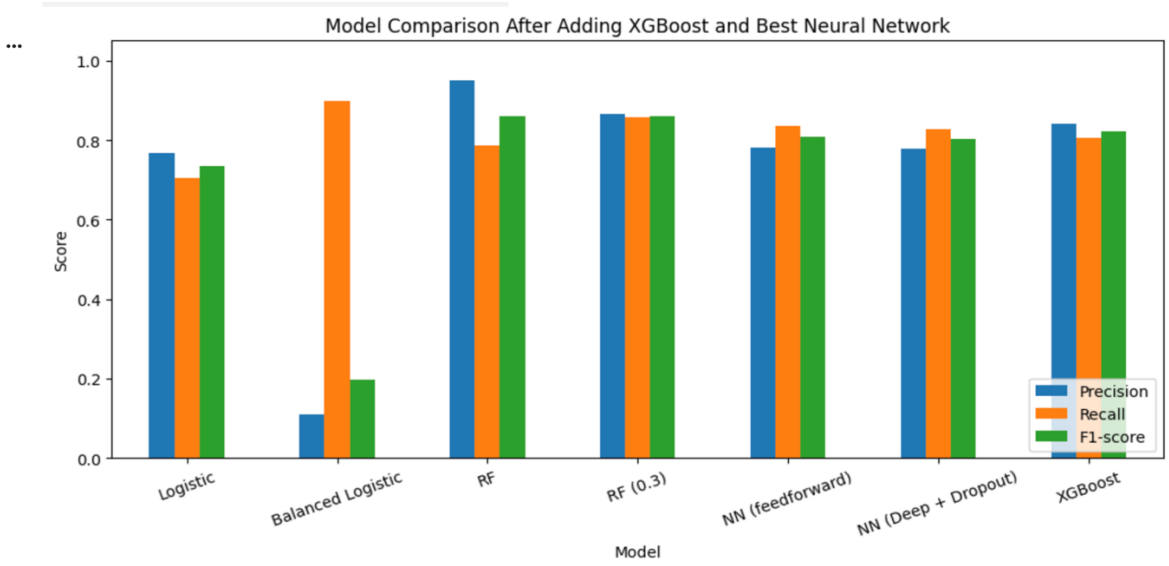
Confusion matrix for the final Random Forest model using threshold 0.3.

10. Results and Comparative Analysis

10.1 Vertical Progression: Results Across Project Stages

The project progressed from simple baseline models to more advanced tuned models.

Model	Precision	Recall	F1-score
Logistic Regression	0.7667	0.7041	0.7340
Balanced Logistic Regression	0.1103	0.8980	0.1964
Random Forest	0.9506	0.7857	0.8603
Random Forest Threshold 0.3	0.8660	0.8571	0.8615
Feedforward Neural Network	0.7810	0.8367	0.8079
Deep Neural Network + Dropout	0.7788	0.8265	0.8020
XGBoost	0.8404	0.8061	0.8229



Final model comparison using precision, recall, and F1-score.

Key observations:

- Logistic Regression worked as a reasonable baseline but had lower recall.
- Balanced Logistic Regression achieved very high recall but very low precision.
- Random Forest performed very strongly even before threshold tuning.
- Random Forest with threshold 0.3 achieved the highest F1-score.
- XGBoost performed strongly but did not beat the tuned Random Forest.
- Neural networks became competitive after architecture comparison but did not outperform tree-based models.

10.2 Horizontal Comparison: Baseline vs Advanced Model

The models were also compared by family.

Model Family	Best Example	Strength	Weakness
Baseline	Logistic Regression	Simple and interpretable	Lower recall
Imbalance-aware baseline	Balanced Logistic Regression	High recall	Too many false positives
Tree-based	Random Forest 0.3	Best F1-score	Less interpretable than Logistic Regression

Model Family	Best Example	Strength	Weakness
Boosting	XGBoost	Strong PR-AUC and ranking	More tuning-sensitive
Neural Network	Feedforward / Deep + Dropout	Flexible architectures	Did not outperform tree-based models

This comparison showed that advanced tree-based models performed best on this tabular fraud detection problem. Neural networks were useful to compare, but they did not provide the strongest final performance.

10.3 Final Model Performan

The final selected model was:

Random Forest with threshold 0.3

Final performance:

Metric	Value
Precision	0.8660
Recall	0.8571
F1-score	0.8615

The confusion matrix for this final model was:

	Predicted Legitimate	Predicted Fraud
Actual Legitimate	56,851	13
Actual Fraud	14	84

This means the model correctly detected 84 out of 98 fraud cases in the test set while producing only 13 false alarms among legitimate transactions.

11. Streamlit Demonstration Application

A Streamlit application was developed to demonstrate the final fraud detection model.

The app allows users to:

- upload a CSV file,
- preview uploaded transaction data,
- check required feature columns,
- run fraud prediction,

- view predicted fraud counts,
- preview predicted fraud cases,
- and download the full prediction results.

The app uses the saved Random Forest model, scaler, feature names, and threshold.

```
# Save best model
joblib.dump(rf_model, "fraud_app_files/random_forest_fraud_model.joblib")

# Save scaler
joblib.dump(scaler, "fraud_app_files/scaler.joblib")

# Save feature names
with open("fraud_app_files/feature_names.json", "w") as f:
    json.dump(list(X_train.columns), f)

# Save selected threshold
with open("fraud_app_files/threshold.json", "w") as f:
    json.dump({"threshold": 0.3}, f)

print("Random Forest model, scaler, feature names, and threshold saved successfully.")
```

When a CSV file is uploaded, the app removes the `Class` column if it exists, reorders the input columns to match the saved feature list, scales the data, predicts fraud probabilities, and applies the threshold of 0.3.

Important app prediction logic:

```
data_for_prediction = data_for_prediction[feature_names]
data_scaled = scaler.transform(data_for_prediction)

fraud_probabilities = model.predict_proba(data_scaled)[:, 1]
predictions = (fraud_probabilities >= threshold).astype(int)
```

This ensures that the app uses the same preprocessing and prediction logic as the notebook.

12. What Worked vs. What Did Not Work

What Worked

Random Forest with threshold tuning worked best overall. It achieved the highest F1-score and gave the best balance between precision and recall.

Threshold tuning was very important. The default threshold did not always produce the best fraud detection behavior. Adjusting the threshold helped control the tradeoff between catching fraud and reducing false alarms.

PR-AUC and F1-score were useful metrics because they focused more on the minority fraud class than accuracy.

The Streamlit app successfully demonstrated the trained model using uploaded transaction data.

Neural network architecture comparison was also useful because it showed how model depth and dropout affected performance.

What Did Not Work as Well

Balanced Logistic Regression achieved high recall but produced too many false positives, which caused very low precision.

Neural networks did not outperform the tree-based models. Although the neural network models became competitive, Random Forest still performed better on this structured tabular dataset.

The anonymized PCA features limited interpretability because the original feature meanings were not available.

The full dataset was too large for a smooth live demo, so a smaller 100-row sample CSV was used for the app demonstration.

13. Discussion

Several important insights emerged from this project.

First, imbalanced datasets require careful evaluation. Accuracy alone was not useful because the model could achieve high accuracy by mostly predicting the majority class.

Second, threshold tuning can significantly change model behavior. In fraud detection, the threshold determines how sensitive the model is to fraud. Lower thresholds may catch more fraud but can also create more false positives.

Third, tree-based models performed better than neural networks for this dataset. Random Forest and XGBoost both performed strongly, but Random Forest with threshold 0.3 achieved the best F1-score.

Fourth, neural networks still provided useful comparisons. The feedforward neural network and dropout-based neural network showed that deeper architectures can become competitive, but they did not outperform the final Random Forest model.

Finally, the Streamlit app connected the modeling work to a practical demonstration. It showed how a trained model can be saved, loaded, and used to make predictions on new uploaded transaction data.

14. Limitations and Future Work

Limitations

The project has several limitations:

- The features are anonymized, so interpretation is limited.
- The app expects the same feature format as the training data.
- The model was trained on historical transactions only.
- Fraud patterns can change over time.
- The Streamlit app is a demonstration, not a production banking system.
- Real-world deployment would require security, monitoring, and retraining.

Future Work

Future improvements could include:

- applying SMOTE or other resampling methods,
- testing cost-sensitive learning,
- adding SHAP explainability,
- monitoring model drift over time,
- retraining on newer fraud patterns,
- improving app deployment online,
- and integrating real-time fraud detection.

SHAP explainability would be especially useful because the dataset features are anonymized. Even though the original meaning of each feature is hidden, SHAP could still help explain which features most influenced a fraud prediction.

15. Conclusion

This project demonstrated how machine learning and neural networks can be applied to a real-world imbalanced fraud detection problem.

The project compared multiple models, including Logistic Regression, Balanced Logistic Regression, Random Forest, XGBoost, and neural networks. Because the dataset was highly imbalanced, evaluation focused on precision, recall, F1-score, and PR-AUC instead of accuracy alone.

The best-performing model was Random Forest with threshold 0.3. It achieved the highest F1-score and provided the best balance between detecting fraudulent transactions and limiting false alarms.

The final Streamlit app demonstrated how the trained model could be used in practice by allowing users to upload transaction data and view fraud predictions.

Overall, the project showed that fraud detection requires threshold-aware evaluation, careful model comparison, and metrics designed for imbalanced classification.

16. References

Kaggle Credit Card Fraud Detection Dataset
https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

Scikit-learn Logistic Regression Documentation
https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Scikit-learn RandomForestClassifier Documentation
https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

Scikit-learn Model Evaluation Documentation
https://scikit-learn.org/stable/modules/model_evaluation.html

Scikit-learn Precision-Recall Curve Documentation
https://scikit-learn.org/stable/auto_examples/model_selection/plot_precision_recall.html

XGBoost Documentation
https://xgboost.readthedocs.io/

TensorFlow Keras Documentation
https://www.tensorflow.org/guide/keras

Streamlit Documentation
https://docs.streamlit.io/

Joblib Documentation
https://joblib.readthedocs.io/

Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G.
Calibrating Probability with Undersampling for Unbalanced Classification. IEEE Symposium Series on Computational Intelligence, 2015.